

ML Engineering Practice Labs

Unit Testing • ML Code Validation • Experiment Tracking • Model Registry

A set of four basic, hands-on labs introducing core engineering practices used in production machine learning workflows.

Prerequisites	Python 3.9+, pip, basic command line, basic Python & ML familiarity
Tools Used	pytest, great_expectations / pandas, MLflow, MLflow Model Registry
Duration	Approx. 1 - 1.5 hours per lab

Table of Contents

Lab 1	Unit Testing for ML Code	3
Lab 2	ML Code & Data Validation	6
Lab 3	Experiment Tracking with MLflow	9
Lab 4	Model Registry with MLflow	12

LAB 1

Unit Testing for ML Code

Writing and running automated tests for data and model code with pytest

Objective	Learn to write unit tests for ML pipeline functions (data preprocessing, feature engineering, model helper functions) using pytest.
Tools	Python 3, pytest
Duration	60-75 minutes

1.1 Background

Unit testing verifies that individual, isolated pieces of code (a single function or class method) behave as expected. In ML systems, unit tests are commonly applied to data-cleaning functions, feature transformations, custom loss/metric functions, and utility code — not to the statistical performance of the trained model itself (that is covered in Lab 2). Good unit tests are fast, deterministic, and independent of each other.

1.2 Setup

1. Create a project folder and a virtual environment.
2. Install pytest.
3. Create the folder structure shown below.

```
mkdir ml_testing_lab && cd ml_testing_lab
python3 -m venv venv
source venv/bin/activate      # venv\Scripts\activate on Windows
pip install pytest pandas numpy

mkdir src tests
touch src/__init__.py src/preprocessing.py tests/test_preprocessing.py
```

1.3 Write the Source Code

Add the following functions to **src/preprocessing.py**:

```
import pandas as pd
import numpy as np

def fill_missing_with_mean(df: pd.DataFrame, column: str) -> pd.DataFrame:
    df = df.copy()
    df[column] = df[column].fillna(df[column].mean())
    return df

def normalize_column(df: pd.DataFrame, column: str) -> pd.DataFrame:
    df = df.copy()
    col = df[column]
    if col.max() == col.min():
        raise ValueError('Cannot normalize a constant column')
    df[column] = (col - col.min()) / (col.max() - col.min())
    return df

def encode_labels(labels: list) -> dict:
    unique = sorted(set(labels))
    return {label: idx for idx, label in enumerate(unique)}
```

1.4 Write the Unit Tests

Add the following to **tests/test_preprocessing.py**:

```
import pandas as pd
import pytest
from src.preprocessing import fill_missing_with_mean, normalize_column, encode_labels

def test_fill_missing_with_mean():
    df = pd.DataFrame({'x': [1, 2, None, 4]})
    result = fill_missing_with_mean(df, 'x')
    assert result['x'].isnull().sum() == 0
    assert result['x'][2] == pytest.approx(2.333, rel=1e-2)

def test_normalize_column_range():
    df = pd.DataFrame({'x': [10, 20, 30, 40]})
    result = normalize_column(df, 'x')
    assert result['x'].min() == 0.0
    assert result['x'].max() == 1.0

def test_normalize_column_constant_raises():
    df = pd.DataFrame({'x': [5, 5, 5]})
    with pytest.raises(ValueError):
        normalize_column(df, 'x')

def test_encode_labels():
    labels = ['cat', 'dog', 'cat', 'bird']
    mapping = encode_labels(labels)
    assert mapping == {'bird': 0, 'cat': 1, 'dog': 2}

@pytest.mark.parametrize('values,expected_min,expected_max', [
    ([1, 2, 3], 0.0, 1.0),
    ([-5, 0, 5], 0.0, 1.0),
])
def test_normalize_parametrized(values, expected_min, expected_max):
    df = pd.DataFrame({'x': values})
    result = normalize_column(df, 'x')
    assert result['x'].min() == expected_min
    assert result['x'].max() == expected_max
```

1.5 Run the Tests

```
pytest -v
pytest --cov=src --cov-report=term-missing # requires: pip install pytest-cov
```

Confirm all tests pass, then intentionally break **normalize_column** (e.g. remove the constant-column check) and re-run pytest to observe a failing test — this demonstrates that the test suite actually catches regressions.

1.6 Exercises

- Write a test for a new function **clip_outliers(df, column, lower, upper)** that clips values to a given range.
- Add a fixture (**@pytest.fixture**) that returns a reusable sample DataFrame for multiple tests.
- Add a negative test that checks a **KeyError** is raised when an invalid column name is passed.
- Generate an HTML coverage report using **pytest-cov** and identify any untested lines.

1.7 Expected Outcome

A working **tests/** suite with at least 6 passing test cases, and the ability to explain how pytest fixtures, parametrization, and exception assertions work.

#	Viva / Discussion Question
1	What is the difference between a unit test and an integration test in an ML pipeline?
2	Why should unit tests avoid depending on a trained model or external data source?
3	What does pytest.approx do, and why is it needed for floating-point assertions?
4	How does test coverage relate to code quality? Is 100% coverage always desirable?

LAB 2

ML Code & Data Validation

Validating data schemas, data quality, and model sanity checks

Objective	Validate input data quality/schema and add basic sanity checks on model behavior before deployment.
Tools	Python 3, pandas, scikit-learn, (optional) great_expectations
Duration	60-75 minutes

2.1 Background

ML code validation has two layers: (1) **data validation** — checking that incoming data matches an expected schema, ranges, and quality thresholds; and (2) **model validation** — sanity-checking that a trained model behaves reasonably (e.g. predictions fall in a valid range, the model beats a trivial baseline, and it is not overtly biased on simple slices of data). This lab covers both, using lightweight, testable Python functions.

2.2 Setup

```
pip install pandas numpy scikit-learn pytest
mkdir -p src tests
touch src/data_validation.py src/model_validation.py tests/test_validation.py
```

2.3 Data Validation Checks

Add to `src/data_validation.py`:

```
import pandas as pd

EXPECTED_SCHEMA = {
    'age': 'int64',
    'income': 'float64',
    'category': 'object',
}

def validate_schema(df: pd.DataFrame, schema: dict = EXPECTED_SCHEMA) -> list:
    errors = []
    for col, dtype in schema.items():
        if col not in df.columns:
            errors.append(f'Missing column: {col}')
        elif str(df[col].dtype) != dtype:
            errors.append(f'{col}: expected {dtype}, got {df[col].dtype}')
    return errors

def validate_ranges(df: pd.DataFrame) -> list:
    errors = []
    if (df['age'] < 0).any() or (df['age'] > 120).any():
        errors.append('age out of valid range [0, 120]')
    if (df['income'] < 0).any():
        errors.append('income cannot be negative')
    return errors

def validate_missing(df: pd.DataFrame, max_missing_frac: float = 0.05) -> list:
    errors = []
    for col in df.columns:
        frac = df[col].isnull().mean()
```

```

        if frac > max_missing_frac:
            errors.append(f'{col} has {frac:.1%} missing values (limit {max_missing_frac:.0%})')
    ')
    return errors

def run_all_checks(df: pd.DataFrame) -> list:
    return validate_schema(df) + validate_ranges(df) + validate_missing(df)

```

2.4 Model Sanity Checks

Add to **src/model_validation.py**:

```

import numpy as np

def check_prediction_range(preds, low=0.0, high=1.0) -> bool:
    return bool(np.all(preds >= low) and np.all(preds <= high))

def check_beats_baseline(model_accuracy: float, baseline_accuracy: float, margin: float = 0.02) -> bool:
    """Trained model must beat a trivial baseline (e.g. majority class) by a margin."""
    return model_accuracy >= baseline_accuracy + margin

def check_no_nan_predictions(preds) -> bool:
    return not np.isnan(preds).any()

def check_invariance(model, x_sample, perturb_fn, tolerance=0.05):
    """Predictions should not change drastically under an irrelevant perturbation."""
    original = model.predict_proba([x_sample])[0]
    perturbed = model.predict_proba([perturb_fn(x_sample)])[0]
    return bool(np.abs(original - perturbed).max() <= tolerance)

```

2.5 Write Tests

Add to **tests/test_validation.py**:

```

import pandas as pd
import numpy as np
from src.data_validation import run_all_checks
from src.model_validation import check_prediction_range, check_beats_baseline

def test_valid_dataframe_passes():
    df = pd.DataFrame({'age': [25, 40], 'income': [50000.0, 62000.0], 'category': ['a', 'b']})
    assert run_all_checks(df) == []

def test_invalid_age_detected():
    df = pd.DataFrame({'age': [-5, 200], 'income': [1.0, 2.0], 'category': ['a', 'b']})
    errors = run_all_checks(df)
    assert any('age' in e for e in errors)

def test_prediction_range():
    preds = np.array([0.1, 0.5, 0.99])
    assert check_prediction_range(preds) is True
    assert check_prediction_range(np.array([1.5])) is False

def test_model_beats_baseline():
    assert check_beats_baseline(0.91, 0.85) is True
    assert check_beats_baseline(0.86, 0.85) is False

pytest -v tests/test_validation.py

```

2.6 Exercises

- Add a duplicate-row check that flags datasets with more than 1% duplicate rows.

- Add a label-balance check that warns if any class represents less than 5% of the dataset.
- (Optional) Install **great_expectations** and re-implement the range/missing checks as Expectations.
- Wire **run_all_checks** into a small script that exits with a non-zero status code if any check fails, so it can be used as a CI gate.

#	Viva / Discussion Question
1	Why validate data schema and ranges separately from model accuracy?
2	What is an invariance test, and why is it useful for detecting bias or brittleness?
3	Why must a model be compared against a baseline rather than judged on accuracy alone?
4	How would you integrate these validation checks into a CI/CD pipeline?

LAB 3

Experiment Tracking with MLflow

Logging parameters, metrics, and artifacts across multiple training runs

Objective	Track and compare multiple model training runs (parameters, metrics, artifacts) using MLflow Tracking.
Tools	Python 3, mlflow, scikit-learn
Duration	60-75 minutes

3.1 Background

Experiment tracking records everything needed to reproduce and compare a training run: hyperparameters, evaluation metrics, the code/data version, and output artifacts such as trained model files or plots. MLflow Tracking provides a local server and Python API for logging this information, plus a UI for comparing runs side by side.

3.2 Setup

```
pip install mlflow scikit-learn pandas
mkdir experiment_tracking_lab && cd experiment_tracking_lab
```

3.3 Write a Tracked Training Script

Save as **train.py**:

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment('wine-classification')

def run_experiment(n_estimators, max_depth):
    X, y = load_wine(return_X_y=True)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    with mlflow.start_run():
        mlflow.log_param('n_estimators', n_estimators)
        mlflow.log_param('max_depth', max_depth)

        model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth, random_state=42)
        model.fit(X_train, y_train)
        preds = model.predict(X_test)

        acc = accuracy_score(y_test, preds)
        f1 = f1_score(y_test, preds, average='macro')
        mlflow.log_metric('accuracy', acc)
        mlflow.log_metric('f1_macro', f1)

        mlflow.sklearn.log_model(model, 'model')
        print(f'n_estimators={n_estimators}, max_depth={max_depth} -> acc={acc:.3f}, f1={f1:.3f}')
```

```
if __name__ == '__main__':
    for n_est in [50, 100, 200]:
        for depth in [3, 5, None]:
            run_experiment(n_est, depth)
```

3.4 Run the Experiments

```
python train.py
mlflow ui # opens on http://127.0.0.1:5000 by default
```

Open the MLflow UI in a browser, select the **wine-classification** experiment, and use the compare-runs view to sort by **accuracy** and **f1_macro**. Identify the best-performing hyperparameter combination.

3.5 Log Additional Artifacts

Extend the script to log a confusion-matrix plot as an artifact:

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

fig, ax = plt.subplots()
ConfusionMatrixDisplay.from_predictions(y_test, preds, ax=ax)
fig.savefig('confusion_matrix.png')
mlflow.log_artifact('confusion_matrix.png')
```

3.6 Exercises

- Add **mlflow.log_metric** calls for precision and recall alongside accuracy and F1.
- Use **mlflow.set_tag** to tag each run with a developer name and a data-version identifier.
- Use the MLflow Python client (**mlflow.search_runs**) to programmatically fetch the run with the highest **f1_macro**.
- Compare two or more runs in the UI and export the comparison as a CSV.

3.7 Expected Outcome

At least 9 logged runs visible in the MLflow UI, each with parameters, metrics, and a saved model artifact, and the ability to identify the best run using both the UI and the Python API.

#	Viva / Discussion Question
1	What is the difference between a parameter, a metric, and an artifact in MLflow?
2	Why is reproducibility important in experiment tracking, and what information is needed to reproduce a run?
3	How does an MLflow 'experiment' differ from a 'run'?
4	How could experiment tracking be integrated with version control (e.g. Git commit hash) for full traceability?

LAB 4

Model Registry with MLflow

Registering, versioning, and promoting models through lifecycle stages

Objective	Register trained models to a central Model Registry, manage versions, and move models through Staging and Production stages.
Tools	Python 3, mlflow (Tracking + Model Registry)
Duration	45-60 minutes

4.1 Background

A model registry is a centralized store for trained model artifacts that adds versioning, stage transitions (e.g. None -> Staging -> Production -> Archived), and metadata/annotations on top of raw model files. It provides one source of truth that separates the question "which model performed best in experiments" (Experiment Tracking, Lab 3) from "which model is currently deployed" (Model Registry).

4.2 Prerequisite

Complete Lab 3 first (or re-run **train.py**) so that at least one run with a logged model exists in the local MLflow tracking store.

4.3 Register a Model

Save as **register_model.py**:

```
import mlflow
from mlflow.tracking import MlflowClient

client = MlflowClient()
experiment = client.get_experiment_by_name('wine-classification')

# Get the best run by accuracy
runs = client.search_runs(
    experiment_ids=[experiment.experiment_id],
    order_by=['metrics.accuracy DESC'],
    max_results=1,
)
best_run = runs[0]
print('Best run id:', best_run.info.run_id, 'accuracy:', best_run.data.metrics['accuracy'])

model_uri = f'runs:{best_run.info.run_id}/model'
registered = mlflow.register_model(model_uri, 'wine-classifier')
print('Registered as version:', registered.version)
```

```
python register_model.py
```

Open the MLflow UI and click the **Models** tab. Confirm that **wine-classifier** appears with version 1 pointing at the selected run.

4.4 Add Version Metadata

```
client.update_model_version(
    name='wine-classifier',
    version=registered.version,
```

```

        description='RandomForest baseline, tuned on n_estimators/max_depth grid.',
    )
    client.set_model_version_tag(
        name='wine-classifier', version=registered.version,
        key='validation_status', value='passed',
    )

```

4.5 Promote Through Stages

MLflow supports stage transitions such as **Staging**, **Production**, and **Archived** (newer MLflow versions use aliases; the classic stage API is shown here for clarity):

```

client.transition_model_version_stage(
    name='wine-classifier',
    version=registered.version,
    stage='Staging',
)
print('Model moved to Staging')

# After manual/automated validation in Staging:
client.transition_model_version_stage(
    name='wine-classifier',
    version=registered.version,
    stage='Production',
    archive_existing_versions=True,  # auto-archive any prior Production version
)
print('Model promoted to Production')

```

4.6 Load and Serve the Production Model

```

import mlflow.pyfunc

model = mlflow.pyfunc.load_model('models:/wine-classifier/Production')
sample_predictions = model.predict(X_test[:5])
print(sample_predictions)

```

Because the code refers to the stage (**Production**) rather than a specific run id, the underlying model version can be swapped later without changing any downstream code.

4.7 Exercises

- Register a second model version from a different run and compare it against version 1 in the UI.
- Promote the second version to **Staging**, then to **Production** with `archive_existing_versions=True`, and confirm version 1 is automatically archived.
- Write a small script that rejects promotion to Production if the run's logged **accuracy** metric is below a threshold (e.g. 0.9).
- Use `client.get_latest_versions('wine-classifier', stages=['Production'])` to fetch the current production version programmatically.

4.8 Expected Outcome

A registered model with at least two versions, one promoted to **Production**, and a working script that loads and predicts using the `models:/wine-classifier/Production` URI.

#	Viva / Discussion Question
1	What problem does a model registry solve that experiment tracking alone does not?

2	Why is it useful for downstream code to reference a stage (e.g. Production) instead of a specific run id?
3	What governance or approval steps might sit between the Staging and Production transitions in a real organization?
4	How would you roll back to a previous model version if the current Production version starts failing?

End of Lab Manual — ML Engineering Practice Labs (Unit Testing, ML Code Validation, Experiment Tracking, Model Registry).